

BERECHENBARKEIT UND KOMPLEXITÄT

Übungsblatt 7

Prof. Rossmanith
C. Grüne, T. Janßen, T. Koglin
Lehrstuhl für Informatik 1
RWTH Aachen

WS 24/25
03.12.2024
Abgabe: Di. 10.12., 16:30

- Die Übungsblätter müssen in Gruppen von 3-4 Studierenden **aus dem gleichen Tutorium** abgegeben werden. Abgaben von Gruppen anderer Größen werden mit 0 Punkten bewertet.
- Die abgegebenen Lösungen müssen mit Namen und Matrikelnummern aller Teammitglieder beschriftet werden.
- Um zur Klausur zugelassen zu werden, muss folgendes erreicht werden: 50% der Punkte in den Blätter 1-6 **und** 50% der Punkte in den Blätter 7-12..
- Es wird nur eine einzelne PDF-Datei als Abgabe akzeptiert. Eingescannte Lösungen sind erlaubt, solange sie gut lesbar sind.
- Die Lösungen zu diesem Blatt werden Di, 10.12, in der Globalübung präsentiert.
- Thema der dieswöchigen Minutests: Definition Einfacher LOOP-Programme.

Tutoraufgabe 7.1

Zeigen Sie, dass folgende Befehle durch ein LOOP-Programm simuliert werden können:

- (a) $x_i := x_j \dot{-} 1$ (modifizierte Vorgängerfunktion mit Ergebnis 0, falls $x_j = 0$)
- (b) $x_i := x_j \dot{-} x_k$ (modifizierte Subtraktion mit Ergebnis 0, falls $x_j < x_k$)
- (c) $x_i := \min\{x_j, x_k\}$
- (d) IF $x_i = c$ THEN P_1 ELSE P_2 ENDIF (Für eine Konstante $c \in \mathbb{N}$)
- (e) $x_i := x_j \text{ DIV } x_k$ (Division ohne Rest, gegeben $x_k > 0$)
- (f) $x_i := x_j \text{ MOD } x_k$ (Modulo, gegeben $x_k > 0$)

Tutoraufgabe 7.2

Wir betrachten das LOOP-Programm P :

```
1: LOOP  $x_1$  DO
2:   LOOP  $x_2$  DO
3:     LOOP  $x_3$  DO
4:        $x_4 := x_4 + 1$ 
5:     ENDLOOP
6:   ENDLOOP
7: ENDLOOP
```

- (a) Bestimmen Sie eine natürliche Zahl m_P , sodass $F_P(n) < A(m_P, n)$ für alle $n \in \mathbb{N}$ gilt.
- (b) Zeigen Sie, dass die Wachstumsfunktion $F_P : \mathbb{N} \rightarrow \mathbb{N}$ des LOOP-Programms P die Beziehung $F_P(n) = \Theta(n^3)$ erfüllt.

Hausaufgabe 7.1

(6 Punkte)

- (a) (3 Punkte) Betrachten Sie die Fibonacci-Zahlen, die wie folgt definiert sind:

$$\begin{aligned} F(1) &= 1, \\ F(2) &= 1, \\ F(k) &= F(k-1) + F(k-2), \quad k \in \mathbb{N}_{>2}. \end{aligned}$$

Geben Sie ein LOOP-Programm an, das eine natürliche Zahl $k \in \mathbb{N}_{>0}$ als Input erhält und die k -te Fibonacci-Zahl ausgibt. Beschreiben Sie außerdem Ihr Programm in 3-5 Sätzen.

Sie dürfen alle Makros aus der Vorlesung und den Tutoraufgaben verwenden.

- (b) (3 Punkte) Gegeben sei das folgende LOOP-Programm P .

```

1:   $x_1 := x_1 + 1;$ 
2:  LOOP  $x_2$  DO
3:     $x_0 := x_0 + 1;$ 
4:  ENDLOOP
5:  LOOP  $x_3$  DO
6:    LOOP  $x_4$  DO
7:       $x_0 := x_0 + 1;$ 
8:    ENDLOOP
9:  ENDLOOP
10:  $x_2 := x_2 + 1;$ 

```

Bestimmen Sie eine natürliche Zahl m_P , sodass $F_P(n) < A(m_P, n)$ für alle $n \in \mathbb{N}$ gilt.

Hausaufgabe 7.2

(4 Punkte)

Wir betrachten nun eine Einschränkung an WHILE-Programme. Die *Schachtelungstiefe* eines WHILE-Programmes ist die maximale Anzahl geschachtelter Schleifen. Dabei ist eine Schleife sowohl als LOOP-Schleife als auch als WHILE-Schleife zu verstehen. Zum Beispiel hat das LOOP-Programm aus Aufgabe 7.1 b) Schachtelungstiefe 2.

Zeigen oder widerlegen Sie: WHILE-Programme mit Schachtelungstiefe 1 (d.h. Schleifen sind erlaubt, aber keine geschachtelten Schleifen) sind turingvollständig.

Hausaufgabe 7.3

(10 Punkte)

In dieser Aufgabe möchten wir uns mit der Mächtigkeit von LOOP und WHILE-Programmen beschäftigen. Wie Sie aus der Vorlesung wissen, ist die Ackermannfunktion nicht LOOP-berechenbar, jedoch WHILE-berechenbar. Dies wollen wir uns nun mit einer Ackermann-ähnlichen Funktion, nämlich der "Up-Arrow-Funktion" B anschauen.

(a) (5 Punkte) Zuerst definieren wir die Funktion $B(m, n, k)$ wie folgt:

$$B(m, n, 1) = m + n \quad (1)$$

$$B(m, n, 2) = m \cdot n \quad (2)$$

$$B(m, n, 3) = m \uparrow n = m^n \quad (3)$$

$$B(m, n, k) = m \underbrace{\uparrow \dots \uparrow}_{k-2} n = \underbrace{m \uparrow \dots \uparrow (m \uparrow \dots \uparrow (\dots (m \uparrow \dots \uparrow m) \dots))}_{k-3} \quad k \geq 4 \quad (4)$$

In der vierten Gleichung wird die vorige Operation rechts-assoziativ $n - 1$ mal angewendet. Zum Beispiel ist $B(m, n, 4)$ ein Potenzturm, der n mal m enthält, also wird $n - 1$ mal potenziert. Beispielsweise kommt in dem Ausdruck $m^2 = m \cdot m$ kommt eine Multiplikation vor.

Sei nun $k \in \mathbb{N}_{>0}$ eine Konstante. Geben Sie für jede Konstante $k \in \mathbb{N}$ ein LOOP-Programm an, das $B(m, n, k)$ für alle $m, n \in \mathbb{N}$ berechnet. Sie dürfen dabei folgende syntaktische Komponenten verwenden:

- Variablen
- Konstanten ($c \in \{0, 1\}$)
- Symbole ($+$, $:=$, $;$)
- Schlüsselwörter (LOOP, DO, ENDLOOP)

Sie dürfen keine weiteren Makros verwenden, die Sie nicht selbst definieren. Erklären Sie ihr Programm in 3-5 Sätzen.

(b) (5 Punkte) Als nächstes definieren wir die Funktion $B(m, n, k)$ in einer rekursiven Weise, für $m, n \in \mathbb{N}_0$ und $k \in \mathbb{N}_{>0}$.

$$B(m, 0, 1) = m \quad n = 0, k = 1 \quad (1)$$

$$B(m, 0, 2) = 0, \quad n = 0, k = 2 \quad (2)$$

$$B(m, 0, k) = 1, \quad n = 0, k \geq 3 \quad (3)$$

$$B(m, n, 1) = B(m + 1, n - 1, 1) \quad k = 1 \quad (4)$$

$$B(m, n, k) = B(m, B(m, n - 1, k), k - 1), \quad \text{sonst} \quad (5)$$

Geben Sie ein WHILE-Programm mit einer konstanten Schachtelungstiefe der Schleifen an, das $B(m, n, k)$ für alle $m, n \in \mathbb{N}_0$ und $k \in \mathbb{N}_{>0}$ berechnet. Um die Aufgabe zu vereinfachen, haben Sie drei Stacks mit den Operationen $push(x \in \mathbb{N})$ (kein Rückgabewert) und $pop()$ (Rückgabewert $y \in \mathbb{N}$) zur Verfügung, um die Werte der drei Parameter m, n, k zu speichern.

Vergewissern Sie sich, dass man Stacks in WHILE-Programmen als Makro programmieren kann, indem Sie sich Folien 38 bis 42 in "VL-09: LOOP und WHILE Programme I" anschauen. Man kann dabei den Stack wie das Band für die Turing-Maschine programmieren. Zum "Pushen" kann man die Operation Multiplikation verwenden, um den

Stackinhalt "nach unten zu drücken", und die Operation Addition, um eine Zahl auf die erste Position zu schreiben. Zum "Poppen" kann man die Operation Modulo benutzen, um Zahlen vom Stack zu lesen, und Division um die Zahlen vom Stack zu löschen. Insbesondere ist der Stack leer, wenn er den Wert 0 besitzt.

Sie dürfen also folgende syntaktische Komponenten verwenden:

- Variablen
- Konstanten ($c \in \{0, 1\}$)
- Symbole ($+$, $:=$, $;$, $\neq$)
- Schlüsselwörter (WHILE, DO, ENDWHILE)
- Die Makros aus den Turaufgaben von diesem Blatt
- Drei Stacks (S_1, S_2, S_3) mit Operationen push und pop.

Sie dürfen keine weiteren Makros verwenden, die Sie nicht selbst definieren. Erklären Sie ihr Programm in 4-8 Sätzen.

Hinweis: Überlegen Sie sich, wie man die Rekursion mit den Stacks umsetzen kann. Achten Sie dabei insbesondere darauf, wann welche Werte auf welchem Stack abgelegt werden müssen.

Aufgabe:	1	2	3	Total
Punkte:	6	4	10	20
Erreicht:				

Abgabefrist: Die Lösungen müssen bis **Di. 10.12., 16:30** im Moodle-Lernraum abgegeben werden.